Informatics

The R language and system
with programming examples
Functions

Claudio Sartori
Department of Computer Science and Engineering
claudio.sartori@unibo.it
https://www.unibo.it/sitoweb/claudio.sartori/

Abstraction

The essence of abstractions is preserving information that is relevant in a given context, forgetting information that is irrelevant in that context.

– John V. Guttag

Real world example (i)

How to cook pasta

- Put *enough* water in a pot
- Add salt
- Heat the water to boiling point
- Put *enough* pasta into the boiling water
- Wait for *enough time*
- Drain pasta and add *enough* seasoning

Real world example (ii)

- The previous example needs some specific values for a correct implementation
- The values are related to specific needs:
  - how many persons will eat
  - what kind of pasta
  - what kind of seasoning

Real world example (iii)

- The previous example can be abstracted as follows

Cook (pasta) spaghetti XYZ for 4 persons and add ragu

action

parameter 1

parameter 2

parameter 3

Real world example (iv)

function definition

```
cook_pasta(kind_of_pasta,n_persons,seasoning)
        …
        operating instructions
        …
```

Real world example (v)

function call

```
cook_pasta(kind_of_pasta=spaghetti_XYZ,
                  n_persons=4,
                  seasoning=ragu)
```

or, shortly

```
cook_pasta(spaghetti_XYZ, 4, ragu)
```

function

Function definition

- interface
  - name
  - formal parameters
- instructions
- the formal parameters are *variables* available *inside* the function

Function call

- name(… actual parameters…)
- the actual parameters are *expressions*

e^x with MacLaurin – function version

algo to R prog

```
function ex_mcLaurin
parameters v, prec
returns ex
use t, i
t ← 1
ex ← t
i ← 1
Repeat if the absolute value
                of t/ex >= prec
    t ← t * v / i
    ex ← ex + t
    i ← i + 1
return ex
```

```
# function ex_mcLaurin
ex_mcLaurin <- function(v, prec) {
    t <- 1
    ex <- t
    i <- 1
    while (abs(t / ex) >= prec) {
      t <- t * v / i
      ex <- ex + t
      i <- i + 1
    }
    return(ex) # computed value
               # is passed as result
  } # function ex_mcLaurin end
```

e^x with MacLaurin – function call

named parameters

algo to R prog

```
Input

function call with named parameters

output
```

```
cat("Computation of exponential with ",
    "MacLaurin series\n")

# input

x <- as.double(readline("Insert x "))

p <- as.double(readline(
    "Insert the precision "))

# function call,
# result stored in ex

ex <- ex_mcLaurin(v = x, prec = p)

cat("Exponential of ",x," : ",ex,"\n")
```

parameter assignment

e^x with MacLaurin – function call

positional parameters

algo to R prog

```
Input

function call with positional parameters

output
```

```
cat("Computation of exponential with ",
    "MacLaurin series\n")

# input

x <- as.double(readline("Insert x "))

p <- as.double(readline(
    "Insert the precision "))

# function call,
# x copied into the first parameter
# p copied into the second parameter
# result stored in ex

ex <- ex_mcLaurin(x, p)

cat("Exponential of ",x," : ",ex,"\n")
```

e^x with MacLaurin – function version

more flexibility

- define a *default value* for the prec parameter

```
# function ex_mcLaurin
ex_mcLaurin <-
            function(v, prec = 10E-15) {
    t <- 1
    ex <- t
    i <- 1
    while (abs(t / ex) >= prec) {
      t <- t * v / i
      ex <- ex + t
      i <- i + 1
    }
    return(ex) # computed value
               # is passed as result
  } # function ex_mcLaurin end
```

e^x with MacLaurin – function call

using default

- if the parameter with a default value is not specified, the default value is assumed

```
cat("Computation of exponential with ",
    "MacLaurin series\n")

# input

x <- as.double(readline("Insert x "))

p <- as.double(readline(
    "Insert the precision "))

# function call,
# x copied into the first parameter
# p not specified,
#       the default value is taken
# result stored in ex

ex <- ex_mcLaurin(x)

cat("Exponential of ",x," : ",ex,"\n")
```

Lists

List

- Sequence of elements *of any kind*
- are built with the constructor **`list()`**

```
> l <- list("a", 1, "b", 2)
```

- elements can have a name

```
> person <- list(first_name = "John",
                 phone = "335 234 5678",
                 birthdate = as.Date("1990/12/16"),
                 weight = 78)
```

- first_name, phone, birthdate, weight are the names of the elements of the variable person, that is a list

Using lists

- single elements can be accessed with an index (as vectors) but with [[]] (double square braces)

```
> person[[2]]
[1] "335 234 5678"
```

- single braces returns the pair <name,value>

```
> person[2]
$phone
[1] "335 234 5678"
> person$name
[1] "John"
```

Function returning more than one value

- The return value can be *a list*

```
ex_mcLaurin <- function(v,
                        prec = 10E-15) {
    t <- 1
    ex <- t
    i <- 1
    while (abs(t / ex) >= prec) {
      t <- t * x / i
      ex <- ex + t
      i <- i + 1
    }
    return(list(result = ex,
                iterations = i))
  } # function ex_mcLaurin end
```

computed value and number of iterations are passed as result into a list

Function returning more than one value

calling and showing the result

- The elements of the list are shown

```
# calling program
x <- as.double(readline("Insert x "))
# precision not passed, using default
r <- ex_mcLaurin(x) # the return
value is a list
cat("Exponential of ",x,
    " : ",r$result,
    " - Iterations : ",
    r$iterations,"\n")
```

Saving a for later use

- A function definition can be saved in a separate file
  - from the editor
  - with an R command

```
dump("<function_name>",
     file ="<file_path_name>")
```

- The file can be loaded into the current program

```
source("<file_path_name>")
```

Exercise:
Gini concentration index (aka Gini coefficient)

- given a vector of observations x_i measures the inequality among values of the frequency distribution
  - for example, levels of income
- Gini coefficient zero expresses *perfect equality*, where all values are the same
  - for example, where everyone has the same income
- Gini coefficient one (or 100%) expresses *maximal inequality* among values
  - e.g. 1 is the limit for increasing n where only one person has all the income or consumption, n-1 have none

Wikipedia https://en.wikipedia.org/wiki/Gini_coefficient

Gini concentration index - components

mean absolute difference

- sum of the absolute distances of all the pairs divided by the number of pairs

$$\frac{\sum_{i=1}^{n}\sum_{j=1}^{n}|x_i - x_j|}{n^2}$$

mean

$$\bar{x} = \frac{\sum_{i=1}^{n} x_i}{n}$$

Gini concentration

- The relative mean absolute difference
  - mean absolute distance divided by the mean of the values
  - The minimum value is 0
  - The theoretical upper limit of this formula is 2, therefore, the Gini index is defined dividing this by 2

$$\frac{\sum_{i=1}^{n}\sum_{j=1}^{n}|x_i - x_j|}{2n^2\bar{x}}$$

Gini Concentration Index

X_1	100	100	10	10
X_2	100	101	20	100
X_3	100	102	30	1000
X_4	100	103	40	10000
X_5	100	104	400	100000
gini index	**0**	**0.009803922**	**0.8**	**0.9444694**

The cumulates (for different data sets)

	ds1	cum	ds2	cum	ds3	cum	ds4	cum
X_1	*100*	100	*100*	100	*10*	10	*10*	10
X_2	*100*	200	*101*	201	*20*	30	*100*	110
X_3	*100*	300	*102*	303	*30*	60	*1000*	1110
X_4	*100*	400	*103*	406	*40*	100	*10000*	11110
X_5	*100*	500	*104*	510	*400*	500	*100000*	111110

Exercise: Gini concentration index

- given a vector of observations x_i in non decreasing order
  - e.g. the income of persons, $x_i \leq x_{i+1}$
- compute the relative cumulates in case of uniform distribution P_i
- compute the relative cumulates Q_i

$$G = \frac{\sum_{i=1}^{n-1} P_i - \sum_{i=1}^{n-1} Q_i}{\sum_{i=1}^{n-1} P_i} = 1 - \frac{\sum_{i=1}^{n-1} Q_i}{\sum_{i=1}^{n-1} P_i}$$

Gini concentration index (ii)

- simplified formula, since

$$P_i = \frac{i}{n} \quad \Rightarrow \quad \sum_{i=1}^{n-1} \frac{i}{n} = \frac{1}{n} \sum_{i=1}^{n-1} i = \frac{1}{n} \frac{n(n-1)}{2} = \frac{n-1}{2}$$

- therefore

$$G = 1 - 2 \frac{\sum_{i=1}^{n-1} Q_i}{n-1}$$

Algorithms: Set Operations

Set operations

- given sets *v1* and *v2* compute
  - the intersection
  - the union
  - the difference
  - the test of inclusion: *target* included in *container*
  - given a vector produce the unique elements
- given a vector *v* generate the set of its *unique values*
- it is a good exercise to study their implementation in R
- sets will be represented as vectors
- see the R examples
  https://github.com/bladerun16/Informatics-Stats-and-Maths/tree/main/R_programs/020_set_functions

The original R set functions

```
union(x, y)
intersect(x, y)
setdiff(x, y)
setequal(x, y)
unique(v) # b.t.w. this is much more powerful
        # and has several additional parameters
duplicated(v) # returns a vector of booleans
     # false if element already present in
     # lower indexes
x %in% y
```

Exercise

write the function

`setequal_vec(v1,v2)`

- returns true if and only if v1 and v2 represent the same set
- must work also if any of the input vector is empty